Isomer-Decoy Designs – Adjacent vs Random-Distal

Worked examples across site counts and two modification types, and the FLR-calibration trade-off

Pioneer.jl – prepared for N. Wamsley

2026-07-03

1. The knob: how far is the decoy from the true site?

An isomer decoy moves a modification off its true residue onto a position that **cannot carry that modification** (so it is wrong by construction). The one real design choice is **where**: how far from the true site.

The current generator uses **spacing +/-1** (immediately flanking). That is the *hardest* possible decoy – and the point of this note is that "hardest" is a deliberate, and quite conservative, choice.

Why distance = difficulty (make it quantitative). Two placements differ only on the fragment ions whose cut falls *between* them. Moving a mod by distance `d` changes exactly the `d` b-ions and `d` y-ions that span the gap – so a decoy at distance `d` has ~**2d site-determining ions**:

decoy distance d	# site-determining ions	difficulty
1 (adjacent)	2	hardest – one b, one y
2	4	hard
3	6	medium
far	many	easy – lots of ions disagree

Adjacent decoys differ from the true form in only **two** ions; a distal decoy differs in many, so the deconvolution rejects it easily. That is the whole story.

2. Two designs

- **Adjacent (current):** decoy site = true site `+/- spacing` (default 1), nearest non-acceptor. Hardest, most conservative.
- **Random-distal (proposed):** decoy site = a **randomly chosen** position on the peptide that is **not an acceptor of that same modification** (and not already modified). Samples a range of distances -> a more diverse null.

Constraint in both: the decoy site cannot be another acceptor of the *same* mod (else it would be a legitimate alternative localization, not a decoy). With multiple mod types, a phospho decoy *may* sit on an M or K (non-phospho residues), just not on another S/T/Y – as long as that residue isn't already modified.

3. Worked examples

Notation: mod strings are `(pos,residue,Unimod)`. Phospho = `Unimod:21` on S/T/Y; Oxidation = `Unimod:35` on M. Acceptors underlined in the sequence.

3.1 One phospho site

Peptide `L M A S G D V K` (len 8). Phospho acceptors: **S4**. (M2 is an oxidation site, shown for the two-mod case.)

- **True:** `(4,S,Unimod:21)`
- **Adjacent decoys** (d=1, sites 3 or 5): `(3,A,Unimod:21)` or `(5,G,Unimod:21)` – 2 site-determining ions (b4, y4).
- **Random-distal draws** (any non-STY in {1,2,3,5,6,7,8}): `(7,V,Unimod:21)` (d=3, 6 ions) . `(1,L,Unimod:21)` (d=3) . `(6,D,Unimod:21)` (d=2).

3.2 Two phospho sites (di-phospho)

Peptide `S A P M T G D V K` (len 9). Phospho acceptors: **S1, T5**.

- **True:** `(1,S,Unimod:21)(5,T,Unimod:21)`
- **Move one site to a random non-acceptor** (not S1/T5):
  - move S1: `(5,T,Unimod:21)(7,D,Unimod:21)` (S1 -> D7)
  - move T5: `(1,S,Unimod:21)(8,V,Unimod:21)` (T5 -> V8)
- **Adjacent version:** move S1->A2 `(2,A..)(5,T..)`, or T5->G6 `(1,S..)(6,G..)`.

One decoy per real by moving a single site (1:1 balance); or one per site if you want a denser null (more columns – mind the deconvolution conditioning).

3.3 Three phospho sites

Peptide `S P A M T A D Y K` (len 9). Phospho acceptors: **S1, T5, Y8**.

- **True:** `(1,S,Unimod:21)(5,T,Unimod:21)(8,Y,Unimod:21)`
- **Move Y8 to a random non-acceptor** (not S1/T5/Y8): `(1,S..)(5,T..)(7,D,Unimod:21)`
- **Adjacent:** Y8 -> D7 or K9; T5 -> A6 or M4; S1 -> P2.

With 3 true sites the acceptors are spread across the peptide, so *real* wrong localizations already span a range of distances – an argument for matching that range rather than always testing d=1 (Section 4).

3.4 Two modification types (phospho + oxidation)

Peptide `M S A P T L R` (len 7). Phospho acceptors: **S2, T5**. Oxidation: **M1**.

- **True peptidoform:** `(1,M,Unimod:35)(2,S,Unimod:21)` – oxidation on M1, phospho on S2.
- **Phospho decoy** (move phospho off S2 to a non-STY, not the oxidized M1; candidates {3,4,6,7}): `(1,M,Unimod:35)(4,P,Unimod:21)` (S2 -> P4). Oxidation kept.
- **Oxidation decoy** (move oxidation off M1 to a non-M, not the phosphorylated S2; candidates {3,4,5,6,7}): `(2,S,Unimod:21)(6,L,Unimod:35)` (M1 -> L6). Phospho kept.

Each modification gets its **own** decoy null (phospho decoys calibrate phospho FLR, oxidation decoys calibrate oxidation FLR). The cross-constraint – a decoy of one mod can't land on a residue already carrying the other – avoids nonsensical double-modified positions.

4. The trade-off this exposes (why it matters)

A target-decoy FLR is only unbiased if the **decoy score distribution matches the score distribution of the *real false* localizations.** Real false localizations are wrong-acceptor calls (S/T/Y -> another S/T/Y), at whatever distances the acceptors happen to sit. So:

- **Adjacent (d=1) decoys are the hardest case.** If real false localizations are a mix of distances, an all-adjacent null is *harder* than average -> it **over**- estimates FLR (conservative). Good for a safe upper bound, bad if you want the true rate.
- **Random-distal decoys are, on average, easier** (bigger d, more disagreeing ions) -> the deconvolution rejects them readily -> **under**-estimates FLR (optimistic). Dangerous if reported as *the* FLR.
- **The unbiased target is to match the acceptor-spacing distribution** – draw the decoy distance from the empirical distribution of gaps between same-mod acceptors on that peptide (or across the library). Then decoys have the same difficulty mix as real wrong-acceptor calls.

Spectrally, a decoy at position `d` behaves exactly like a *hypothetical real isomer* at `d` (same site-determining-ion shifts) – the residue identity doesn't change the physics, only the guarantee that it's wrong. So "difficulty" is purely about distance, and the design is really "what distance distribution do I sample?"

5. Recommendation

- Keep the current **`spacing`** parameter as the difficulty knob – it already spans d=1 (hardest) upward.
- Add a **`random`** mode: decoy distance drawn per site, either uniform over valid non-acceptor positions (the proposed design) or, better, **sampled to match the same-mod acceptor-spacing distribution** (the least-biased option).
- Consider a **mix** (some adjacent, some distal) so the null spans the full difficulty range the real errors occupy.
- Because the standards give us **ground-truth FLR**, we can *calibrate the choice empirically*: pick the decoy design whose decoy-estimated FLR best matches the known FLR on the standards, then apply it library-wide. That turns this from a guess into a fit.
- Reproducibility: seed the RNG (e.g. from precursor_idx) so a rebuild is identical.

Bottom line. Adjacent (d=1) is the conservative extreme; random-distal is the optimistic extreme; the honest estimator matches the real acceptor-spacing. The `spacing` knob already exists; adding a distance *distribution* (and validating it against the standards' known FLR) is the principled next step.